

LAST RESORT · HOTEL COMPLEX

Hotel Management Database System

Bulat Iglamov, Jasmine Zhu, Yucheng Ming, Elshadie Solomon

A full-stack hotel operations portal — tracking properties, rooms, guests, reservations, stays, billing & services and more.

Python 3

Flask

SQLite

Jinja

HTML-CSS



What We Built



Hotel Management Portal

A web-based operations console for front-desk staff. Browse 3 Park Avenue hotel properties, drill into buildings, wings, individual rooms and their live availability.



Normalized SQLite Database

20+ relational tables enforcing referential integrity. Covers the full hotel lifecycle: properties → rooms → guests → reservations → stays → billing → services.



Real-time Room Search

POST-based search across 6 columns simultaneously — hotel name, building, wing, room type, number and status — using SQL parameterized queries.

3

Hotel Properties

20+

Database Tables

5

Flask Routes

7

HTML Templates

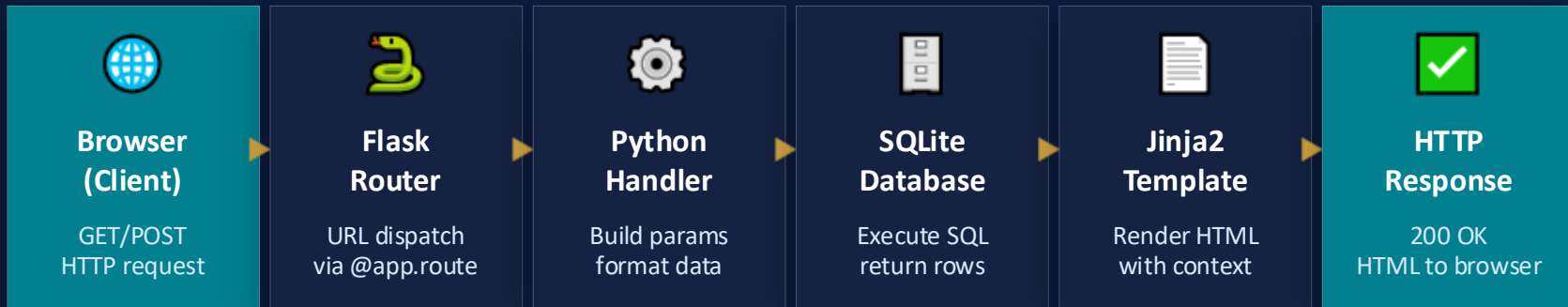
4

Guest Records

3

Junction Tables

How a Request Flows



Project File Structure

app.py	Flask application — all routes & business logic	templates/	7 Jinja2 HTML files (base, splash, index, hotel, room, search, error)
create.sql	DDL — all 20+ CREATE TABLE statements	static/css/	main.css (dark editorial theme) + splash.css
insert.sql	Seed data for hotels, rooms, guests, stays	last_resort.db	Pre-populated SQLite database file

Website Page Walkthrough

Splash Page

/ (splash)

- Luxury marketing landing page
- Full-bleed background image
- Single CTA → Enter portal
- Separate splash.css for distinct feel

Dashboard

/home

- Live stats bar (hotels, rooms, guests)
- Hotel property cards with counts
- Active stays table
- Reservations table
- Recent messages panel

Property Page

/hotel?hotelId=N

- Property name & address header
- Full room grid for that hotel
- Rooms grouped by building & wing
- Color-coded status tiles
- Click through to room detail

Room Detail

/room?roomId=N

- Room number, type, floor & rate
- Occupancy & smoking policy
- Wing proximity features
- Amenity feature list with quantities
- Back navigation link

Search Results

/search (POST)

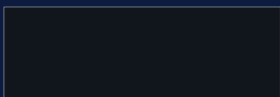
- POST form from nav bar
- LIKE across 6 columns
- Result count in header
- Room tiles same as hotel page
- 'Nothing found' empty state

CSS Theme, Typography & UI Components

Colour Palette



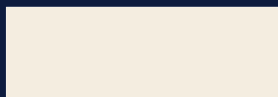
--ink
Background



--ink-2
Panels



--gold
Accent Gold



--paper
Text



--rust
Warning



--moss
Success

Typography

- **Cormorant Garamond (serif)**

Headings h1, h2, brand logotype, hero text, room numbers — italic weighted

- **Inter (sans-serif)**

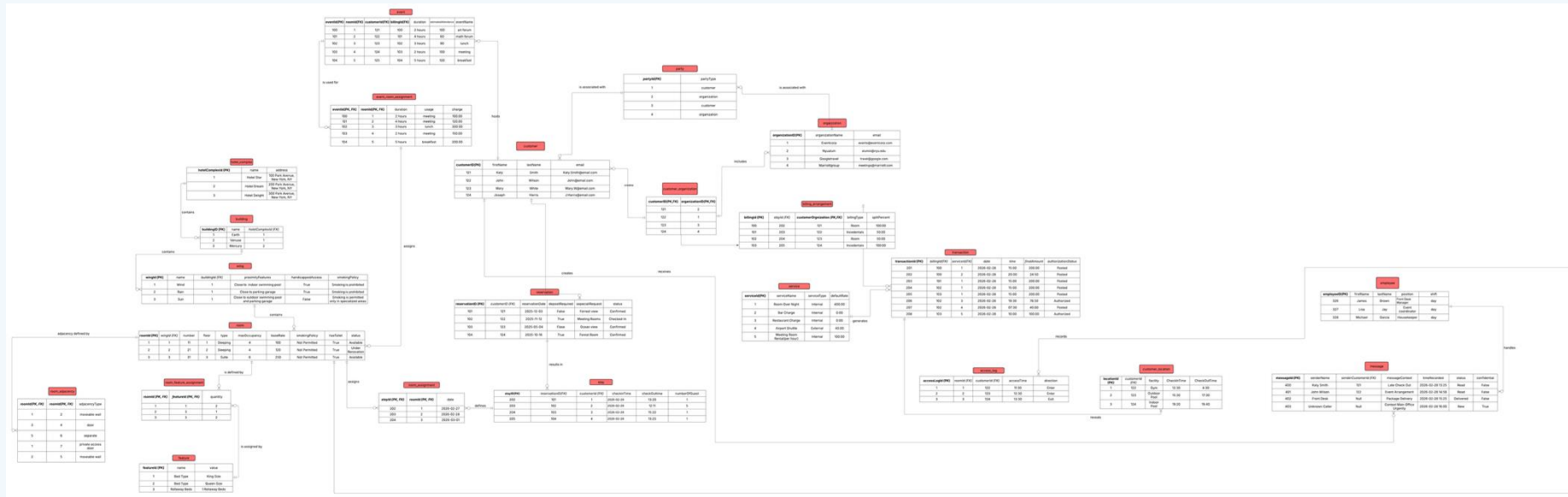
Body copy, nav links, badges, stat labels, table data — weight 300–600

- **SF Mono / Consolas (mono)**

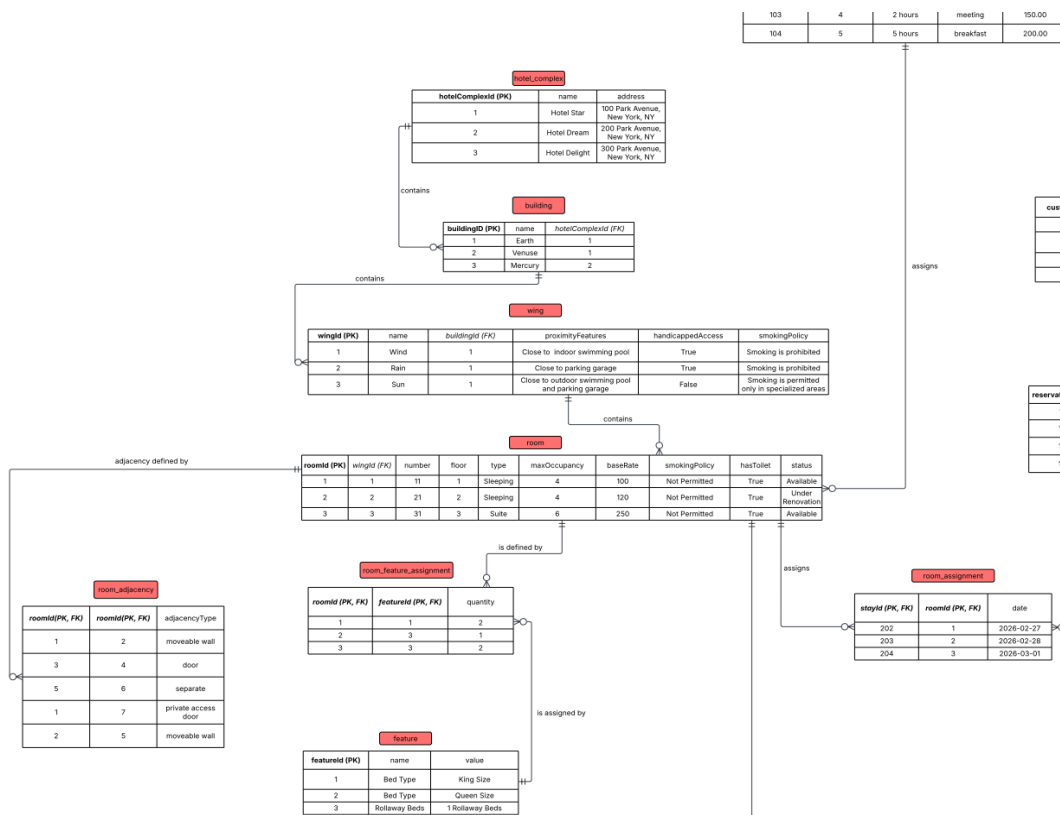
Panel count badges, timestamps, room rates — tabular numerals

UI Components

- Sticky glassmorphism header (backdrop-filter blur)
- Hotel cards — hover lift + gold top-border animation
- Room tiles — left-side status color bar (green/amber/rust)
- Data tables — zebra hover, uppercase headers, monospace IDs
- Messages panel — confidential flag with rust border
- Film-grain CSS texture overlay via ::before pseudo-element



Hotel Complex → Building → Wing → Room



hotel_complex

3 records

Stores 3 Park Ave properties (Star, Dream, Delight). Top of the ownership hierarchy.



building

FK → hotel

Earth, Venus, Mercury. A complex can have multiple buildings. FK: hotelComplexId.



wing

FK → building

Subdivides buildings. Carries smoking policy, handicapped access, proximity features.



room

FK → wing

Leaf node. Stores type, floor, rate, max occupancy, toilet, status, smoking policy.



room_feature_assignment

junction M:M

Junction table: room ↔ feature. Adds quantity (e.g. '2 King Size beds').

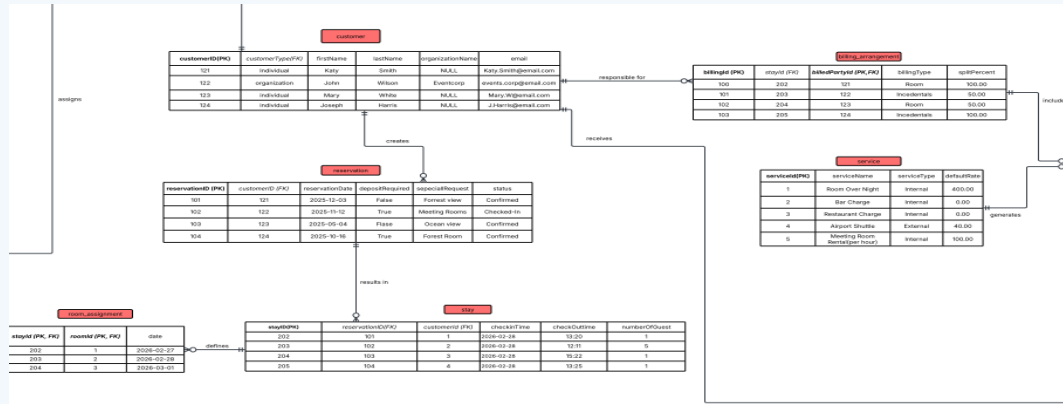


room_adjacency

self-join

Self-referential: two roomIds + adjacencyType (moveable wall, door, private access).

Customer → Reservation → Stay → Room Assignment



customer

4 guests (customerID 121-124). Stores first/last name + email.



organization

Corporate accounts (Eventcorp, NYU Alumni, Google Travel...). Linked via customer_organization.



customer_organization

Junction table: customers belong to one or more organizations. Supports corporate billing.



reservation

The booking. Status: Confirmed / Checked-In. Has deposit flag, special request, date.



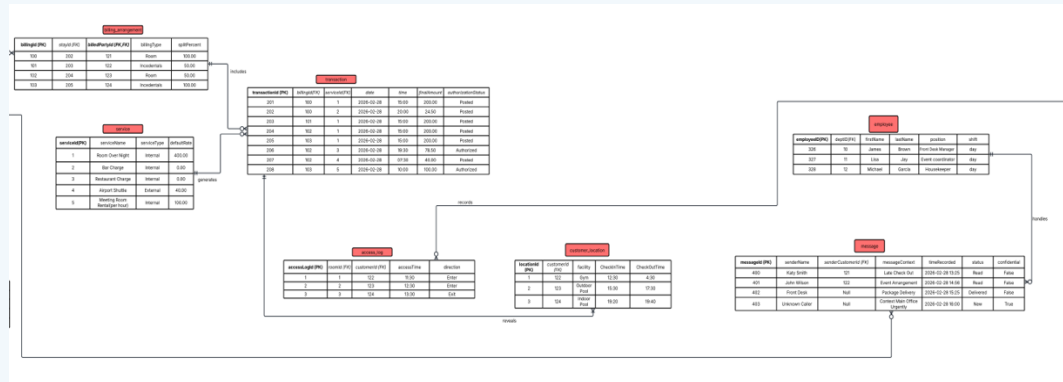
stay

Activated when guest arrives. Links to reservation. Stores check-in/out times, guest count.



room_assignment

Composite PK (stayID, roomId, date). Records which room is used each night of a stay.



Billing, Services & Transactions



employee table — stores employeeid, first/last name, position, and shift. Linked via access_log for room-access auditing.

billing_arrangement

Links a stay to a payer (customer or org). Stores billing type (Room/Incidentals) and split percent — enabling split billing between parties.

service

Catalogue of hotel services: Room Overnight, Bar Charge, Restaurant. Stores type (Internal/External) and default rate.

transaction_info

Individual charges. Links billing → service. Records date, time, final amount, and authorization status (Posted, Pending, Failed).

event + event_room

Event management: named events with attendance estimates assigned to rooms. Separate charge and usage tracking per room.

customer_location

Tracks guest movement through hotel facilities (gym, spa, restaurant) with check-in/out times per visit.

message

Front-desk messaging. Sender name, context, timestamp, read status, and a confidential flag that triggers a special UI treatment.

SQL SCHEMA — CREATE.SQL

```
26 CREATE TABLE hotel_complex (  
27     hotelComplexId INTEGER PRIMARY KEY,  
28     name TEXT NOT NULL,  
29     address TEXT NOT NULL  
30 );  
31  
32 CREATE TABLE building (  
33     buildingId INTEGER PRIMARY KEY,  
34     name TEXT NOT NULL,  
35     hotelComplexId INTEGER NOT NULL,  
36     FOREIGN KEY (hotelComplexId) REFERENCES hotel_complex(hotelComplexId)  
37 );  
38  
39 CREATE TABLE wing (  
40     wingId INTEGER PRIMARY KEY,  
41     name TEXT NOT NULL,  
42     buildingId INTEGER NOT NULL,  
43     proximityFeatures TEXT,  
44     handicappedAccess INTEGER NOT NULL,  
45     smokingPolicy TEXT NOT NULL,  
46     FOREIGN KEY (buildingId) REFERENCES building(buildingId)  
47 );  
48  
49 CREATE TABLE room (  
50     roomId INTEGER PRIMARY KEY,  
51     wingId INTEGER NOT NULL,  
52     number INTEGER NOT NULL,  
53     floor INTEGER NOT NULL,  
54     type TEXT NOT NULL,  
55     maxOccupancy INTEGER NOT NULL,  
56     baseRate DECIMAL(10,2) NOT NULL,  
57     smokingPolicy TEXT NOT NULL,  
58     hasToilet INTEGER NOT NULL,  
59     status TEXT NOT NULL,  
60     FOREIGN KEY (wingId) REFERENCES wing(wingId)  
61 );  
62  
138 CREATE TABLE billing_arrangement (  
139     billingId INTEGER PRIMARY KEY,  
140     stayId INTEGER NOT NULL,  
141     customerId INTEGER NOT NULL,  
142     billingType TEXT NOT NULL,  
143     splitPercent DECIMAL(5,2) NOT NULL,  
144     FOREIGN KEY (stayId) REFERENCES stay(stayId),  
145     FOREIGN KEY (customerId) REFERENCES customer(customerId)  
146 );  
147  
148 CREATE TABLE service (  
149     serviceId INTEGER PRIMARY KEY,  
150     serviceName TEXT NOT NULL,  
151     serviceType TEXT NOT NULL,  
152     defaultRate DECIMAL(10,2) NOT NULL  
153 );  
154  
155 CREATE TABLE transaction_info (  
156     transactionId INTEGER PRIMARY KEY,  
157     billingId INTEGER NOT NULL,  
158     serviceId INTEGER,  
159     date DATE NOT NULL,  
160     time TEXT NOT NULL,  
161     finalAmount DECIMAL(10,2) NOT NULL,  
162     authorizationStatus TEXT NOT NULL,  
163     FOREIGN KEY (billingId) REFERENCES billing_arrangement(billingId),  
164     FOREIGN KEY (serviceId) REFERENCES service(serviceId)  
165 );  
166  
167 CREATE TABLE event (  
168     eventId INTEGER PRIMARY KEY,  
169     duration TEXT NOT NULL,  
170     estimatedAttendance INTEGER,  
171     eventName TEXT NOT NULL  
172 );  
173
```

20 tables total

hotel_complex → building → wing → room hierarchy

FK constraints

PRAGMA foreign_keys = ON enforced throughout

Billing chain

reservation → stay → billing_arrangement → transaction_info

Feature system

Many-to-many: room ↔ feature via room_feature_assignment

Events & access

event_room_assignment & access_log for venue tracking

Customer, Reservation, Stay & Assignment

```

94 CREATE TABLE organization (
95     organizationId INTEGER PRIMARY KEY,
96     organizationName TEXT NOT NULL,
97     email TEXT NOT NULL
98 );
99
100 CREATE TABLE customer_organization (
101     customerId INTEGER NOT NULL,
102     organizationId INTEGER NOT NULL,
103     PRIMARY KEY (customerId, organizationId),
104     FOREIGN KEY (customerId) REFERENCES customer(customerId),
105     FOREIGN KEY (organizationId) REFERENCES organization(organizationId)
106 );
107
108 CREATE TABLE reservation (
109     reservationId INTEGER PRIMARY KEY,
110     customerId INTEGER NOT NULL,
111     reservationDate DATE NOT NULL,
112     depositRequired INTEGER NOT NULL,
113     specialRequest TEXT,
114     status TEXT NOT NULL,
115     FOREIGN KEY (customerId) REFERENCES customer(customerId)
116 );
117
118 CREATE TABLE stay (
119     stayId INTEGER PRIMARY KEY,
120     reservationId INTEGER NOT NULL,
121     customerId INTEGER NOT NULL,
122     checkinTime TEXT NOT NULL,
123     checkoutTime TEXT NOT NULL,
124     numberOfGuest INTEGER NOT NULL,
125     FOREIGN KEY (reservationId) REFERENCES reservation(reservationId),
126     FOREIGN KEY (customerId) REFERENCES customer(customerId)
127 );
128

```



Composite PK — room_assignment

(stayId, roomId, date) — prevents duplicate room bookings on the same night. One stay can span multiple dates with different rooms.



Reservation status lifecycle

Text enum: 'Confirmed' → 'Checked-In' → 'Checked-Out' / 'Cancelled'. Managed at application layer, not via DB constraint.



Stay → Reservation relationship

stay.reservationId links to reservation. The stay.customerId is present but data integrity is enforced via reservation.customerId (by design choice).



Date + Time storage

SQLite stores dates as TEXT format. checkinTime stores HH:MM, checkoutTime stores YYYY-MM-DD — queried via SQLite date functions.

Insert data

```

INSERT INTO hotel_complex (hotelComplexId, name, address) VALUES
(1, 'Hotel Star', '100 Park Avenue, New York, NY'),
(2, 'Hotel Dream', '200 Park Avenue, New York, NY'),
(3, 'Hotel Delight', '300 Park Avenue, New York, NY');

INSERT INTO building (buildingId, name, hotelComplexId) VALUES
(1, 'Earth', 1),
(2, 'Venus', 1),
(3, 'Mercury', 1),
(4, 'Mars', 3),
(5, 'Jupiter', 3),
(6, 'Saturn', 2),
(7, 'Uranus', 1),
(8, 'Neptune', 2);

INSERT INTO wing (wingId, name, buildingId, proximityFeatures, handicappedAccess, smokingPolicy) VALUES
(1, 'Wind', 1, 'Close to indoor swimming pool', 1, 'Smoking is prohibited'),
(2, 'Rain', 1, 'Close to parking garage', 1, 'Smoking is prohibited'),
(3, 'Sun', 4, 'Close to outdoor swimming pool and parking garage', 0, 'Smoking is permitted only in specialized areas'),
(4, 'Fog', 5, 'Far from everything', 0, 'Smoking is permitted only in specialized areas'),
(5, 'Rainbow', 2, 'Prime location', 1, 'Smoking is prohibited'),
(6, 'Aurora', 7, 'Close to everything', 1, 'Smoking is prohibited'),
(7, 'Tide', 3, 'Close to the ocean', 0, 'Smoking is permitted only outside'),
(8, 'Eclipse', 8, 'Forestry location', 1, 'Smoking is prohibited'),
(9, 'Twilight', 6, 'Best view of the sky', 1, 'Smoking is permitted on upper floors');

105 INSERT INTO billing_arrangement (billingId, stayId, customerId, billingType, splitPercent) VALUES
106 (100, 202, 121, 'Room', 100.00),
107 (101, 203, 122, 'Incidentals', 50.00),
108 (102, 204, 123, 'Room', 50.00),
109 (103, 205, 124, 'Incidentals', 100.00);
110
111 INSERT INTO service (serviceId, serviceName, serviceType, defaultRate) VALUES
112 (1, 'Room Overnight', 'Internal', 400.00),
113 (2, 'Bar Charge', 'Internal', 0.00),
114 (3, 'Restaurant Charge', 'Internal', 0.00),
115 (4, 'Airport Shuttle', 'External', 40.00),
116 (5, 'Meeting Room Rental (per hour)', 'Internal', 100.00);
117
118 INSERT INTO transaction_info (transactionId, billingId, serviceId, date, time, finalAmount, authorizationStatus) VALUES
119 (201, 100, 1, '2026-02-28', '15:00', 200.00, 'Posted'),
120 (202, 100, 2, '2026-02-28', '20:00', 24.50, 'Posted'),
121 (203, 101, 1, '2026-02-28', '15:00', 200.00, 'Posted'),
122 (204, 102, 1, '2026-02-28', '15:00', 200.00, 'Posted'),
123 (205, 103, 1, '2026-02-28', '15:00', 200.00, 'Posted'),
124 (206, 102, 3, '2026-02-28', '19:30', 78.50, 'Authorized'),
125 (207, 102, 4, '2026-02-28', '07:30', 40.00, 'Posted'),
126 (208, 103, 5, '2026-02-28', '10:00', 100.00, 'Authorized');

```

Hotels & Buildings

- ▶ 3 hotel complexes
- ▶ 8 buildings
- ▶ 9 wings
- ▶ 16 rooms

Guests & Bookings

- ▶ 4 customers
- ▶ 4 organizations
- ▶ 4 reservations
- ▶ 4 stays

Financial

- ▶ 4 billing records
- ▶ 8 transactions
- ▶ 5 service types

Events & Staff

- ▶ 5 events
- ▶ 5 room assignments
- ▶ 3 employees
- ▶ 4 messages

Room rates range from \$80 (Single Sleeping) → \$300 (Suite Deluxe)

Flask Routes Design

```

3 from flask import Flask, render_template, request, redirect, url_for
4 import os
5 import sqlite3
6
7 app = Flask(__name__)
8 DATABASE = "last_resort.db"
9
10 # Set up the connection
11 def get_db_connection():
12     conn = sqlite3.connect(DATABASE)
13     conn.row_factory = sqlite3.Row
14     return conn
15
16 # The function that is used in the future
17 def get_hotels(conn):
18     return conn.execute("SELECT * FROM hotel_complex ORDER BY name").fetchall()
19
20 # Show splash
21 @app.route("/", methods=["GET"])
22 def splash():
23     return render_template("splash.html")
24

```

```

166 # This is for room page
167 @app.route("/room", methods=["GET"])
168 def room_detail():
169     room_id = request.args.get("roomId", type=int)
170     conn = get_db_connection()
171     hotels = get_hotels(conn)
172
173     room = conn.execute("""
174         SELECT r.*, w.name AS wingName, w.proximityFeatures,
175             b.name AS buildingName, h.name AS hotelName,
176             h.hotelComplexId
177         FROM room r
178         JOIN wing w ON w.wingId = r.wingId
179         JOIN building b ON b.buildingId = w.buildingId
180         JOIN hotel_complex h ON h.hotelComplexId = b.hotelComplexId
181         WHERE r.roomId = ?
182     """, (room_id,)).fetchone()
183
184     features = conn.execute("""
185         SELECT f.name, f.value, rfa.quantity
186         FROM room_feature_assignment rfa
187         JOIN feature f ON f.featureId = rfa.featureId
188         WHERE rfa.roomId = ?
189     """, (room_id,)).fetchall()
190

```

```

103 # This is for hotels page
104 @app.route("/hotels", methods=["GET"])
105 def hotels():
106     conn = get_db_connection()
107     hotels = get_hotels(conn)
108
109     hotel_cards = conn.execute("""
110         SELECT h.hotelComplexId, h.name, h.address,
111             COUNT(DISTINCT b.buildingId) AS buildingCount,
112             COUNT(DISTINCT w.wingId) AS wingCount,
113             COUNT(DISTINCT r.roomId) AS roomCount
114         FROM hotel_complex h
115         LEFT JOIN building b ON b.hotelComplexId = h.hotelComplexId
116         LEFT JOIN wing w ON w.buildingId = b.buildingId
117         LEFT JOIN room r ON r.wingId = w.wingId
118         GROUP BY h.hotelComplexId
119         ORDER BY h.name
120     """).fetchall()
121
122     building_rows = conn.execute("""
123         SELECT b.hotelComplexId, b.name
124         FROM building b
125         ORDER BY b.hotelComplexId, b.name
126     """).fetchall()
127     buildings_by_hotel = {}
128     for row in building_rows:
129         buildings_by_hotel.setdefault(row["hotelComplexId"], []).append(row["name"])
130

```

Route	Template	Purpose
GET /	splash.html	Landing page
GET /home	index.html	Dashboard with stats
GET /hotels	hotels.html	Hotel overview cards
GET /hotel	hotel.html	Single hotel rooms
GET /room	room.html	Room detail & features
GET POST /search	search.html	Smart search engine
GET /search/help	search_help	Search command docs

Standard Handler Pattern

1. conn = get_db_connection()
2. hotels = get_hotels(conn) # nav dropdown
3. Execute page-specific SQL queries
4. conn.close()
5. return render_template(..., **context)

Multi-Table JOINS Powering the Dashboard - Consistency

```

hotel_cards = conn.execute("""
    SELECT h.hotelComplexId, h.name, h.address,
           COUNT(DISTINCT b.buildingId) AS buildingCount,
           COUNT(DISTINCT r.roomId) AS roomCount
    FROM hotel_complex h
    LEFT JOIN building b ON b.hotelComplexId = h.hotelComplexId
    LEFT JOIN wing w ON w.buildingId = b.buildingId
    LEFT JOIN room r ON r.wingId = w.wingId
    GROUP BY h.hotelComplexId
    ORDER BY h.name
""").fetchall()

active_stays = conn.execute("""
    SELECT s.stayId, s.checkinTime, s.checkoutTime, s.numberOfGuest,
           c.firstName, c.lastName,
           r.specialRequest, r.status
    FROM stay s
    JOIN reservation r ON r.reservationId = s.reservationId
    JOIN customer c ON c.customerId = r.customerId
    ORDER BY s.checkinTime
""").fetchall()

rooms = conn.execute("""
    SELECT r.roomId, r.number, r.floor, r.type, r.maxOccupancy,
           r.baseRate, r.status,
           w.name AS wingName,
           b.name AS buildingName,
           h.name AS hotelName
    FROM room r
    JOIN wing w ON w.wingId = r.wingId
    JOIN building b ON b.buildingId = w.buildingId
    JOIN hotel_complex h ON h.hotelComplexId = b.hotelComplexId
    ORDER BY h.name, b.name, r.floor, r.number
""").fetchall()

```

① Hotel Cards — 4-way LEFT JOIN

hotel_complex → building → wing → room.
 LEFT JOIN preserves hotels with 0 rooms.
 COUNT DISTINCT avoids double-counting.

② Active Stays — JOIN through reservation

stay.customerId carries sequential seed IDs 1-4, but customer.customerId is 121-124. Joining via reservation.customerId gives the real guest names.

③ Room Catalogue — 4-table INNER JOIN

Full location path per room: hotel → building → wing → room. ORDER BY creates a human-readable hierarchy. Used in the rooms panel and hotel page.

Advanced Search System - Accuracy

```
def _fuzzy_search(term):
    conn = get_db_connection()
    rooms = conn.execute("""
        SELECT r.*, w.name AS wingName, b.name AS buildingName, h.name AS hotelName
        FROM room r
        JOIN wing w          ON w.wingId = r.wingId
        JOIN building b      ON b.buildingId = w.buildingId
        JOIN hotel_complex h ON h.hotelComplexId = b.hotelComplexId
        WHERE CAST(r.number AS TEXT) LIKE ?
               OR lower(r.type) LIKE lower(?)
               OR lower(r.status) LIKE lower(?)
               OR lower(w.name) LIKE lower(?)
               OR lower(b.name) LIKE lower(?)
               OR lower(h.name) LIKE lower(?)
        ORDER BY h.name, b.name, r.number
        """, tuple(f"%{term}%" for _ in range(6))).fetchall()
    conn.close()
    return _render("rooms", term,
                  rooms=rooms,
                  subtitle=f"{len(rooms)} room{' ' if len(rooms)==1 else 's'} matched.")
```



SQL Injection Safety

Six ? placeholders — never f-string interpolation into SQL. Ensure no SQL command is plugged.



Case-insensitive Matching

lower(col) LIKE lower(?) on text columns. Room number is cast from INT to TEXT first.



Tuple Generator Pattern

tuple(f"%{term}%" for _ in range(6)) generates the same wrapped term for all 6 OR conditions.



Full Location Path

4-table JOIN ensures results show hotelName, buildingName, wingName alongside the matching room — no extra round trips.

HTML Template (Frontend + Dynamic Data)

```
<h1>{{ selected.name }}</h1>
<p class="pageHeadSub">{{ selected.address }}</p>
</section>

<section class="dashboard">
  <div class="panel panel--full">
    <div class="panelHeader">
      <h2>Rooms</h2>
      <span class="panelCount">{{ rooms|length }}</span>
    </div>
    <div class="roomGrid">
      {% for room in rooms %}
        <a class="roomTile roomTile--{{ room.status|lower|replace(' ', '-') }}" href="{{ url_for('room_detail', roomId=room.id) }}">
          <div class="roomTileTop">
            <span class="roomNum">#{{ room.number }}</span>
            <span class="roomType">{{ room.type }}</span>
          </div>
          <div class="roomTileMid">
            <p class="roomLoc">{{ room.buildingName }} · {{ room.wingName }}</p>
            <p class="roomFloor">Floor {{ room.floor }} · Sleeps {{ room.maxOccupancy }}</p>
          </div>
          <div class="roomTileBot">
            <span class="roomRate">${{ '%.0f'|format(room.baseRate) }}<small>/night</small></span>
            <span class="roomStatus">{{ room.status }}</span>
          </div>
        </a>
      {% endfor %}
    </div>
  </div>
</section>
```

```

5 <section class="pageHead">
6   <p class="eyebrow">{{ room.hotelName }} · {{ room.buildingName }} · {{ room.wingName }}</p>
7   <h1>Room {{ room.number }}<span class="pageHeadAccent"> · {{ room.type }}</span></h1>
8   <p class="pageHeadSub">Floor {{ room.floor }} · Sleeps up to {{ room.maxOccupancy }} · ${{ '%.2f'|format(room.baseRate) }} /
9   <p>
10    <span class="badge badge--{{ room.status|lower|replace(' ', '')|replace('-', '') }}">{{ room.status }}</span>
11  </p>
12 </section>
```

① What it does :

- Displays hotel + room information dynamically
- Uses Jinja ({{ }}) to insert backend data

② (Hotel) Key Features:

- Loops through rooms → {% for room in rooms %}
- Shows details (number, type, price, status)
- Updates automatically from database
- Makes the page scalable + dynamic

① What it does

- Displays detailed information about a specific room
- Pulls data dynamically (room number, floor, occupancy, features)
- Uses a base layout to match the rest of the website

② (Room) Key Features

- **Template Inheritance:** Uses {% extends "base.html" %} for consistent design
- **Dynamic Data Rendering:** {{ }} / **Structured Layout:** Sections for header, details, and features
- **Conditional Logic:** Shows values like “Yes/No” depending on data

HTML Template (Frontend + Dynamic Data)

```
<h1>Results for <em>{{ searchTerm }}</em></h1>
{% if subtitle %}<p class="pageHeadSub">{{ subtitle }}</p>{% endif %}
{% endif %}
</section>

<section class="dashboard">
  <div class="panel panel--full">

    {# ----- ROOM RESULTS (fuzzy, status:, rate:, tier:, top-rooms, above-avg) ----- #}
    {% if resultType == "rooms" %}
      {% if rooms|length == 0 %}
        <p class="empty">No rooms matched. Try the <a class="goldLink" href="{{ url_for('search_help') }}">search guide</a>.</p>
      {% else %}
        <div class="roomGrid">
          {% for room in rooms %}
            <a class="roomTile roomTile--{{ room.status|lower|replace(' ', '-') }}" href="{{ url_for('room_detail', roomid=room.roomid) }}">
              <div class="roomTileTop">
                <span class="roomNum">#{{ room.number }}</span>
                <span class="roomType">{{ room.type }}</span>
              </div>
              <div class="roomTileMid">
                <p class="roomLoc">{{ room.hotelName }} · {{ room.buildingName }} · {{ room.wingName }}</p>
                <p class="roomFloor">Floor {{ room.floor }} · Sleeps {{ room.maxOccupancy }}</p>
              </div>
              <div class="roomTileBot">
                <span class="roomRate">${{ '%.0f'|format(room.baseRate) }}<small>/night</small></span>
                <span class="roomStatus">{{ room.status }}</span>
              </div>
            </a>
          {% endfor %}
        </div>
      {% endif %}
    {% endif %}
  </div>
</section>
```

① What it does :

- Displays search results based on user input
- Dynamically updates content depending on the search type
- Shows different results (rooms, reservations, or guests)
- Provides helpful messages when no results or errors occur
- Keeps a consistent layout using the base template

② (Search) Key Features:

- **Template Inheritance:** Uses `{% extends "base.html" %}` for consistent design
- **Dynamic Rendering:** `{{ }}` displays search results (rooms, guests, etc.)
- **Conditional Logic:** `{% if %}` changes what is shown based on `resultType`

Strengths, Weaknesses, and Improvements

Strengths

- Highly normalized database (20+ tables)
- Real-world modeling of hotel operations
- Strong use of JOINS and relational integrity
- Secure and efficient query design
- Clean frontend with intuitive navigation

Weaknesses

- Limited dataset (only a few records)
- No user authentication/login system
- SQLite limits scalability for large systems

Improvements

- Enhance frontend interactivity and analytics
- Separate access for front desk staff, managers, and admins
- Implement authentication and role-based permissions

Thank You

for Listening

Last Resort Hotel Complex

